

Data Link Layer



THE UNIVERSITY OF
WESTERN
AUSTRALIA

CITS3002
Jin Hong

- ❑ The data link layer provides the following key services between the physical and network layers:
 - bundling and unbundling groups of bits into frames
 - ❖ The physical layer only sends raw bits (0s and 1s).
 - ❖ The data link layer groups these bits into structures called *frames*.
 - *controlling* a flow of frames between sender and receiver.
 - ❖ The data link layer ensures the receiver **is not overwhelmed**
 - ❖ It controls the **sending rate**
 - detecting and correcting "higher-level" transmission errors,
 - ❖ problems in the order or delivery of frames, not errors in the bits themselves

- ❑ **MAC** = Media Access Control (aka Medium Access Control)
 - Protocol for **transmitting** and **receiving** frames at the link layer
 - Is responsible for how devices access the **shared communication medium** (e.g., Ethernet cable or Wi-Fi channel).
 - **Detect** (and possibly correct) **errors** in the frame

- ❑ **MAC Address**
 - **Link-layer** address
 - Each network interface has a MAC address, which is a **unique** identifier used at the link layer

Frame

- ❑ Frame is just a simple **container** for network **packets**.
- ❑ Framing is the process of wrapping a **network packet** with **certain information** before sending out.
- ❑ The **network layer** produces a **packet**, and the **data link layer** wraps it into a **frame** before sending out.
- ❑ **Encapsulation** means wrapping the packet with **header** and **trailer** information.

- ❑ A frame typically consists of:
 - Flag: **indication** for the **start** and the **end** of a frame
 - Header: source/destination addresses, as well as other control information
 - **Data** from the upper layer
 - Trailer: error detection/correction code (**CRC**)

Ethernet

- ❑ Mid-1970s: created at Xerox by Bob Metcalfe
 - 2.74 Mbps Ethernet over 9.5 mm thick coaxial cable
- ❑ 1980s
 - Standardized in 1985 as IEEE 802.3
- ❑ 1990:
 - 10BASE-T over twisted pair wiring @ 10 Mbps
- ❑ 1995: Fast Ethernet: 100 Mbps
- ❑ 1999: Gigabit Ethernet: 1 Gbps
- ❑ 2006: 10 Gb Ethernet: 10 Gbps
- ❑ 2010: 100 GbE: 100 Gbps
- ❑ 2017: 400 GbE: 400 Gbps

Ethernet Frame

- ❑ 8 bytes: **preamble (7 bytes)** & start-of-frame **delimiter (1 byte)**
- ❑ **Variable** size data: 42-1500 bytes
 - **No length** field: the transceiver grabs the entire frame
- ❑ **Interframe** gap: at least **96 bit** wait time
 - A silent period where **no** data is transmitted

MAC destination	MAC source	Type	Payload (42-1500 bytes)	CRC
-----------------	------------	------	-------------------------	-----

higher level protocol ID:
0x0800 = IPv4, 0x86DD =
IPv6

Three levels of data link layer

❑ There are **three levels** of data link layer **complexity**:

➤ **Simplex** connectionless:

- ❖ The sender simply sends its frames **without** waiting for the receiver to **acknowledge** them.
- ❖ **No attempt** is made to detect or **re-transmit** lost frames.
- ❖ Most **modern LAN technologies** (such as Ethernet) use this method and **leave** error resolution to higher layers.
- ❖ This is also termed *an unacknowledged connectionless service*.

Three levels of data link layer

➤ Half-duplex connectionless

- ❖ Each frame sent is **individually** acknowledged
- ❖ Frames which are **lost** or garbled are **retransmitted** if the receiver requests them (again) or after a suitable **timeout**.
- ❖ This is also termed an ***acknowledged connectionless service***.



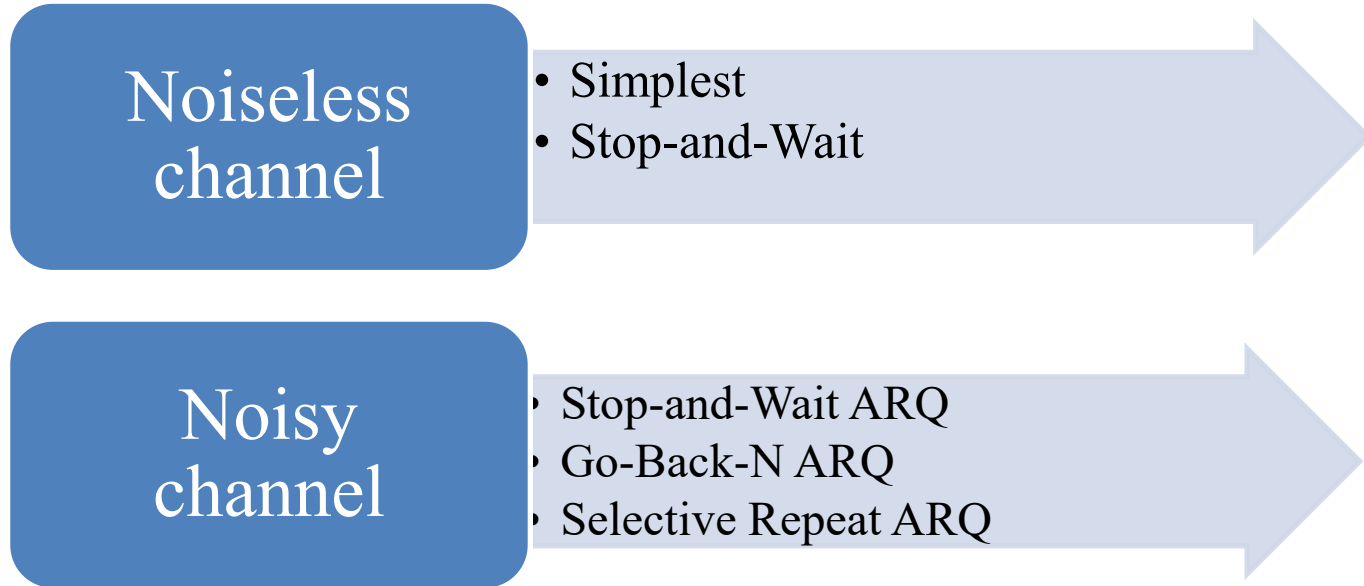
Three levels of data link layer

➤ **Full-duplex** connection-oriented:

- ❖ Each frame is individually **numbered** and is guaranteed by the data link layer to be received once and only once and in the **right order**.
- ❖ The result is that the data link layer presents a **reliable frame stream** to the network layer.
- ❖ This is also termed an **acknowledged connected service**.



Data link layer protocols



We are going to start with the simplest one and build up the protocol features as we go.

Basic Protocol

- ❑ Our early protocols will benefit from all using the same data types, so for simplicity we'll define some **representative** ones first.
- ❑ As our protocols "evolve" we'll need **to distinguish different types of data link frames** from each other.
- ❑ Notice that the frame itself consists of a **header section** and the actual **data** to be sent. We'll need to **extend this header structure** as our protocols develop.
- ❑ We can represent each data link frame as a structure in programming languages that support aliasing with a 'byte' data type, and permit a program to access/copy these bytes using their memory addresses.
- ❑ In Python, we will **pack the frame contents into a sequence of bytes** (and unpack from bytes)

Basic Protocol

```
class Frame:
    MAX_DATA_SIZE = 1000

    def __init__(self, data: bytes = bytes()):
        # firstly, the frame's header
        self.len = len(data)
        if self.len > Frame.MAX_DATA_SIZE:
            raise RuntimeError('Too much data for one Frame!')

        # followed by the payload
        self.data = data
```

Basic Protocol

```
import struct

class Frame:
    ...

    def pack(self):
        return struct.pack('! H 1000s', self.len, self.data)
```

- ❑ It uses Python's `struct.pack()` function to **encode values into a binary format**
- ❑ `!` : use **network byte order**, standard size, no padding (between fields)
- ❑ `H` : an **unsigned two-byte integer** (`self.len`)
- ❑ `1000s` : one thousand bytes (`self.data`)

Basic Protocol

- ❑ Importantly, even though we've defined our frame to have a **fixed maximum size**, we hope to **avoid** sending the **whole** (large) frame.
 - For example, while the **standard Ethernet** frame may carry up to 1500 bytes of data, it may only need to carry, say, 80 bytes.

- ❑ In fact, protocols often exchange frames **consisting of only the header** (e.g. acknowledgment frames).

Basic Protocol

```
import struct

class Frame:
    ...

    def pack(self):
        return struct.pack('! H {}s'.format(self.len),
                           self.len, self.data)
```

- ❑ ! : use network byte order, standard size, no alignment (padding)
- ❑ H : an unsigned two-byte integer (self.len)
- ❑ {}s : (formatted to) exactly self.len bytes (self.data)

Basic Protocol

```
import struct

class Frame:
    ...

    def unpack(self, packed: bytes):
        header_length = struct.calcsize('! H')
        self.len, = struct.unpack_from('!H', packed)
        self.data, = struct.unpack_from('!{}s'.format(self.len),
            packed, header_length)
```

Unrestricted Simplex Protocol

- Assuming:

- ☐ A (unidirectional) **error free** channel
- ☐ The **sender's network** layer has **unlimited data** to send (being "pushed down" from above), and
- ☐ The **receiver's network** layer has an **infinite** buffer to receive the data (being "pushed up" from below).
- ☐ The functions **READ_xxx_LAYER()** and **WRITE_xxx_LAYER()** block until their actions are complete - they execute **synchronously**.

The sender simply sends its frames **without waiting** for the receiver to acknowledge them.

Unrestricted Simplex Protocol

❑ In the sender:

```
frame = Frame()
link = 1

while True:
    frame.data = READ_NETWORK_LAYER()
    frame.len = len(frame.data)
    packed = frame.pack()
    WRITE_PHYSICAL_LAYER(link, packed)
```

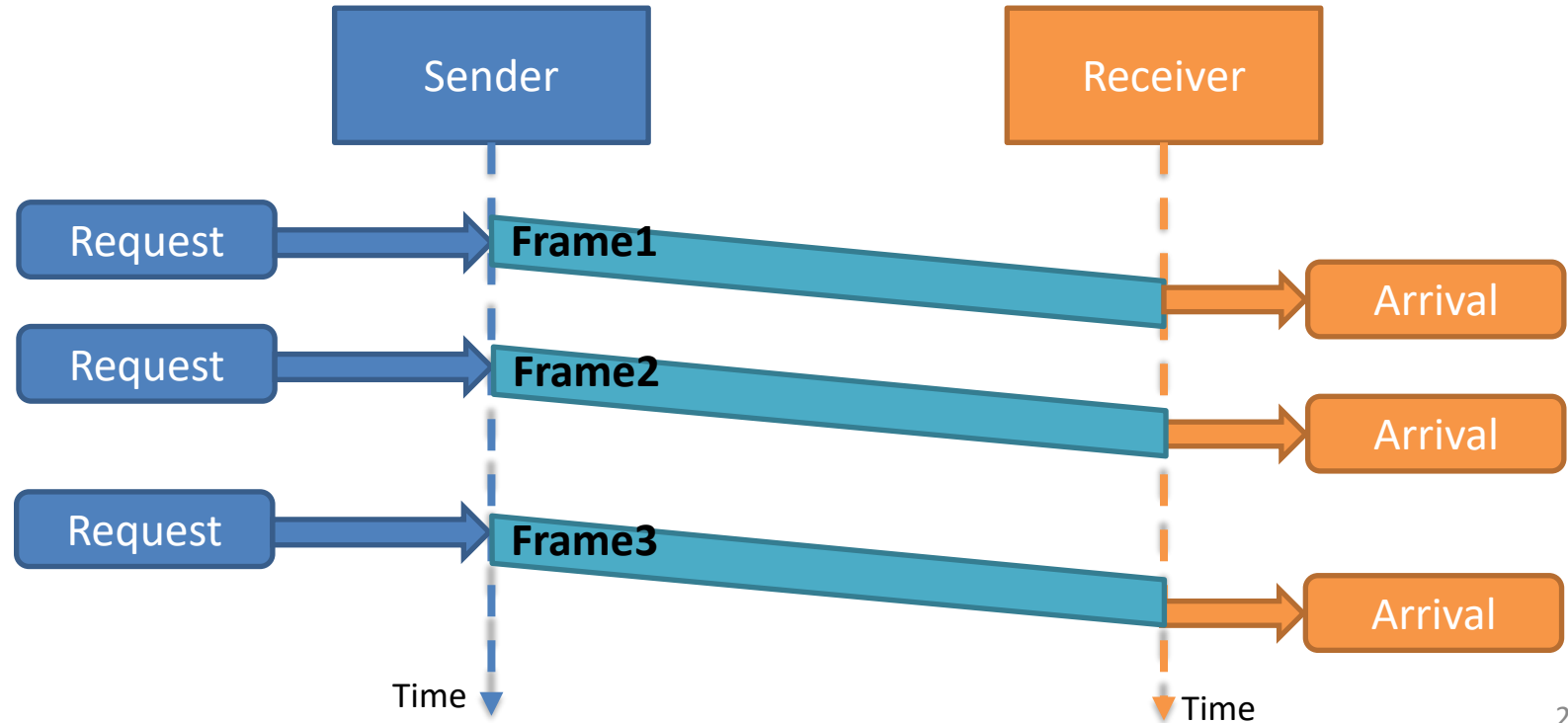
Unrestricted Simplex Protocol

❑ In the receiver:

```
frame = Frame()
link = None

while True:
    link, packed = READ_PHYSICAL_LAYER()
    frame.unpack(packed)
    WRITE_NETWORK_LAYER(frame.data)
```

Unrestricted Simplex Protocol



Half-Duplex Stop-and-Wait Protocol



- ❑ Next, we'll remove the assumption that the receiver can safely **receive** and **store** an infinite amount of data.
- ❑ When this happens, we say that the (fast) sender *floods* the receiver, and the "drowning" receiver needs to **control the rate** at which data is received.
- ❑ We achieve this by sending an **acknowledgement** back to the sender.
- ❑ For now, we'll keep the assumption that the **unidirectional channel is error-free**.

Half-Duplex Stop-and-Wait Protocol

❑ In the sender:

```
frame = Frame()
link = 1

while True:
    frame.data = READ_NETWORK_LAYER()
    frame.len = len(frame.data)
    packed = frame.pack()
    WRITE_PHYSICAL_LAYER(link, packed)
    link, packed = READ_PHYSICAL_LAYER()
```

Now the sender **has to listen for the ACK**

Half-Duplex Stop-and-Wait Protocol

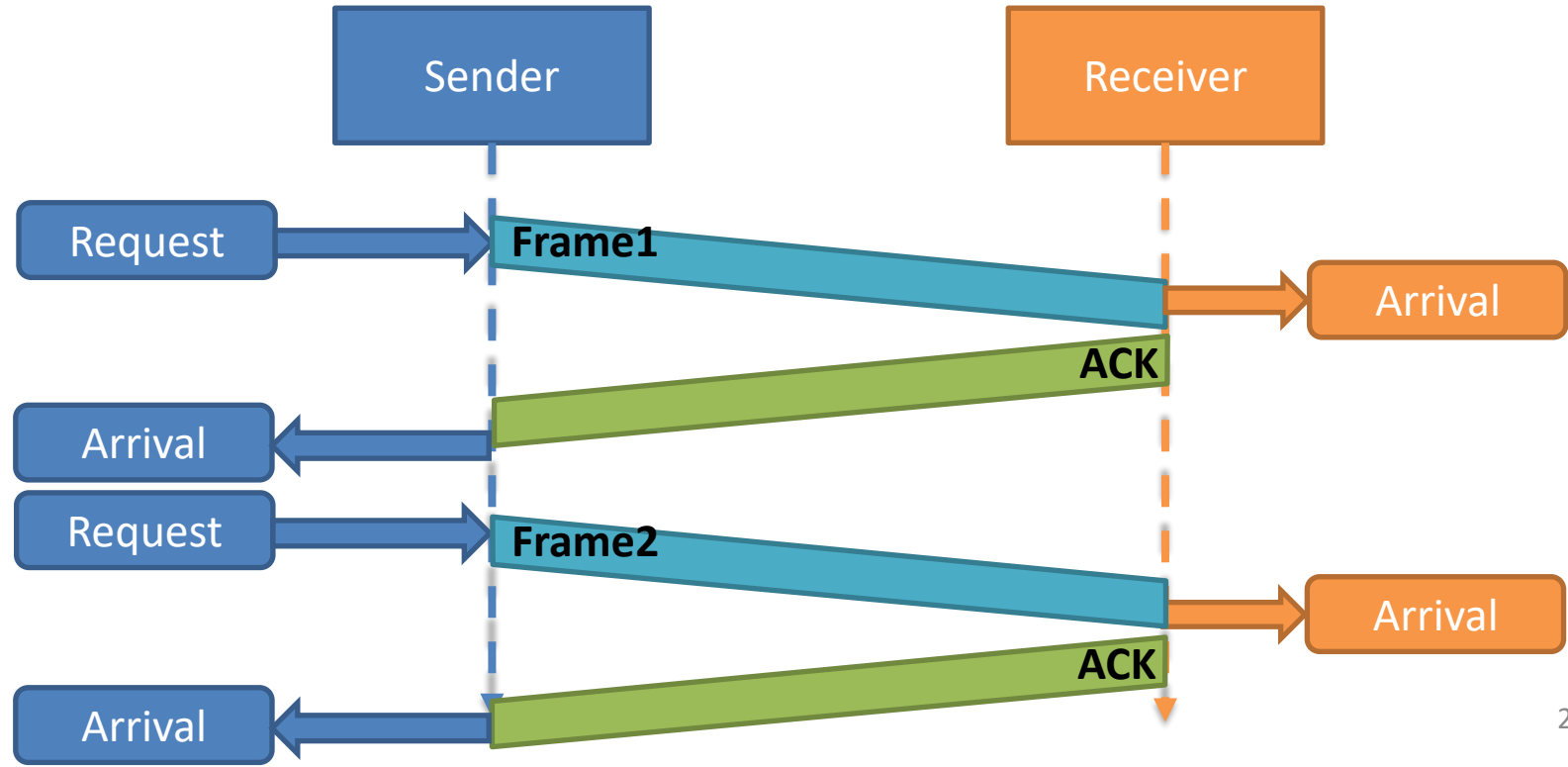
❑ In the receiver:

```
frame = Frame()
link = None

while True:
    link, packed = READ_PHYSICAL_LAYER()
    frame.unpack(packed)
    WRITE_NETWORK_LAYER(frame.data)

    frame.len = 0
    WRITE_PHYSICAL_LAYER(link, frame.pack())
```


Half-Duplex Stop-and-Wait Protocol



Half-Duplex Stop-and-Wait Protocol



- ❑ Adding an acknowledgement can ensure that packets are delivered properly.
- ❑ It also introduces some **problems**
 - What are they?

Detecting Frame Corruption

- ❑ Next, we'll **remove the assumption** that the channel is **error free**; frames (only) may now be corrupted during transmission, introducing the need for checksums.
 - **Similar to CRC, checksum** is used to detect transmission errors by adding a computed value to the data so the receiver can verify its integrity.
 - A checksum is usually a simple arithmetic calculation.
 - We'll also introduce a **FRAMETYPE** to distinguish what a frame is being **used for**.
- ❑ In the receiver we need to ensure that the **checksum as received** is in fact the checksum that the sender ***should have*** calculated.
- ❑ If the two are different, then the frame has been **corrupted**.

Detecting Frame Corruption

```
class FrameType(IntEnum):
    DLL_DATA = 0
    DLL_ACK = 1
    DLL_NACK = 2

class Frame:
    MAX_DATA_SIZE = 1000

    def __init__(self, frametype: FrameType, data: bytes = bytes()):
        # firstly, the frame's header
        self.type = frametype

        self.checksum = 0 # checksum of the whole frame

        self.len = len(data) # length of the payload, only
        if self.len > Frame.MAX_DATA_SIZE:
            raise RuntimeError('Too much data for one Frame!')

        # followed by the payload
        self.data = data
```

Detecting Frame Corruption

```
def pack(self):
    return struct.pack('!HHH{s}'.format(self.len),
                      self.type, self.checksum, self.len, self.data)

def unpack(self, packed: bytes):
    header_length = struct.calcsize('!HHH')

    self.type, self.checksum, self.len = struct.unpack_from('!HHH',
                                                            packed)

    self.data, = struct.unpack_from('!{s}'.format(self.len),
                                    packed, header_length)
```

Detecting Frame Corruption

❑ In the
sender:

```
frame = Frame()
ackframe = Frame()
link = 1

while True:
    frame.data = READ_NETWORK_LAYER()
    frame.type = FrameType.DLL_DATA
    frame.len = len(frame.data)
    frame.checksum = 0
    frame.checksum = crc16(frame.pack())

    while True:
        WRITE_PHYSICAL_LAYER(link, frame.pack())
        acklink, ackpacked = READ_PHYSICAL_LAYER()
        ackframe = ackframe.unpack(ackpacked)
        if ackframe.type == FrameType.DLL_ACK:
            break
```

Detecting Frame Corruption

- ❑ In the receiver:

```
frame = Frame()

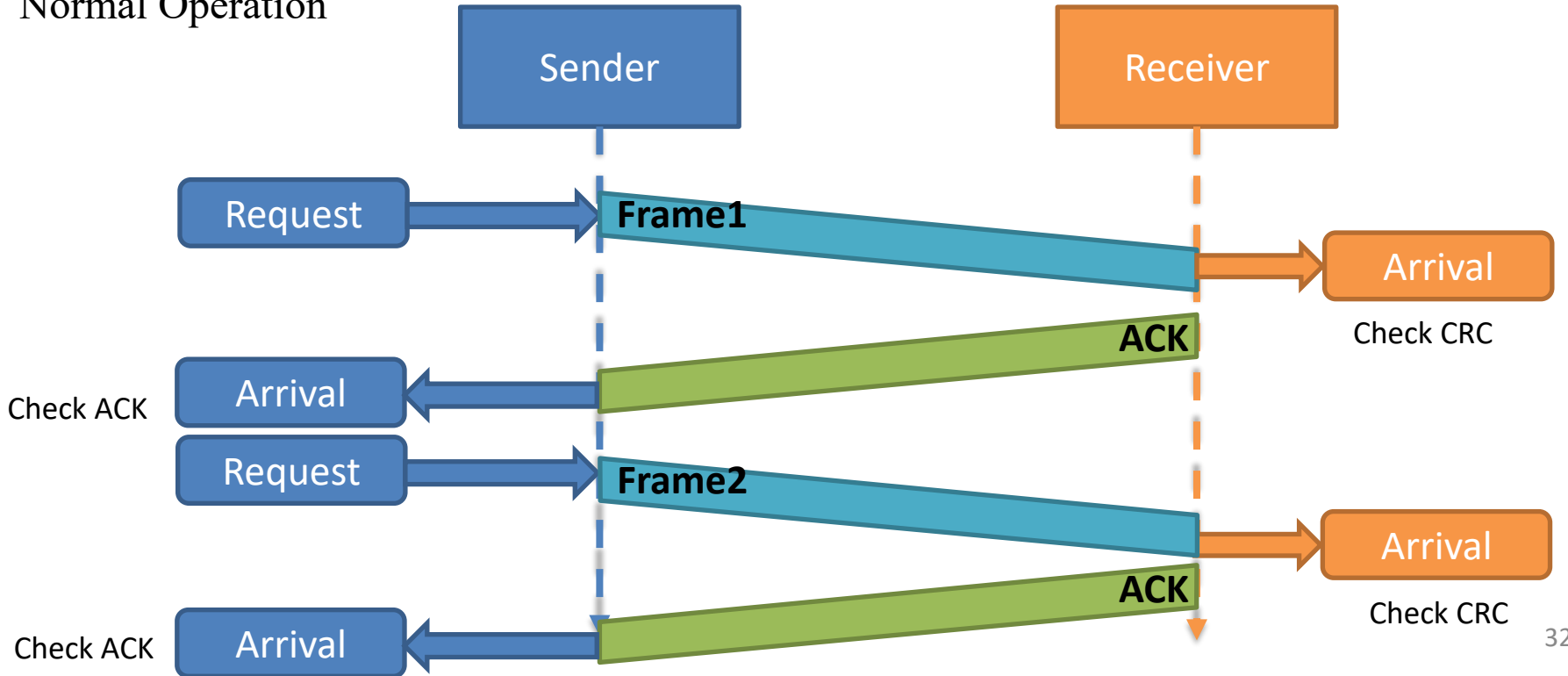
while True:
    link, packed = READ_PHYSICAL_LAYER()
    frame.unpack(packed)
    checksum = frame.checksum
    frame.checksum = 0

    if checksum == crc16(frame.pack()):
        WRITE_NETWORK_LAYER(frame.data)
        frame.type = FrameType.DLL_ACK
    else:
        frame.type = FrameType.DLL_NACK

    frame.len = 0
    WRITE_PHYSICAL_LAYER(link, frame.pack())
```

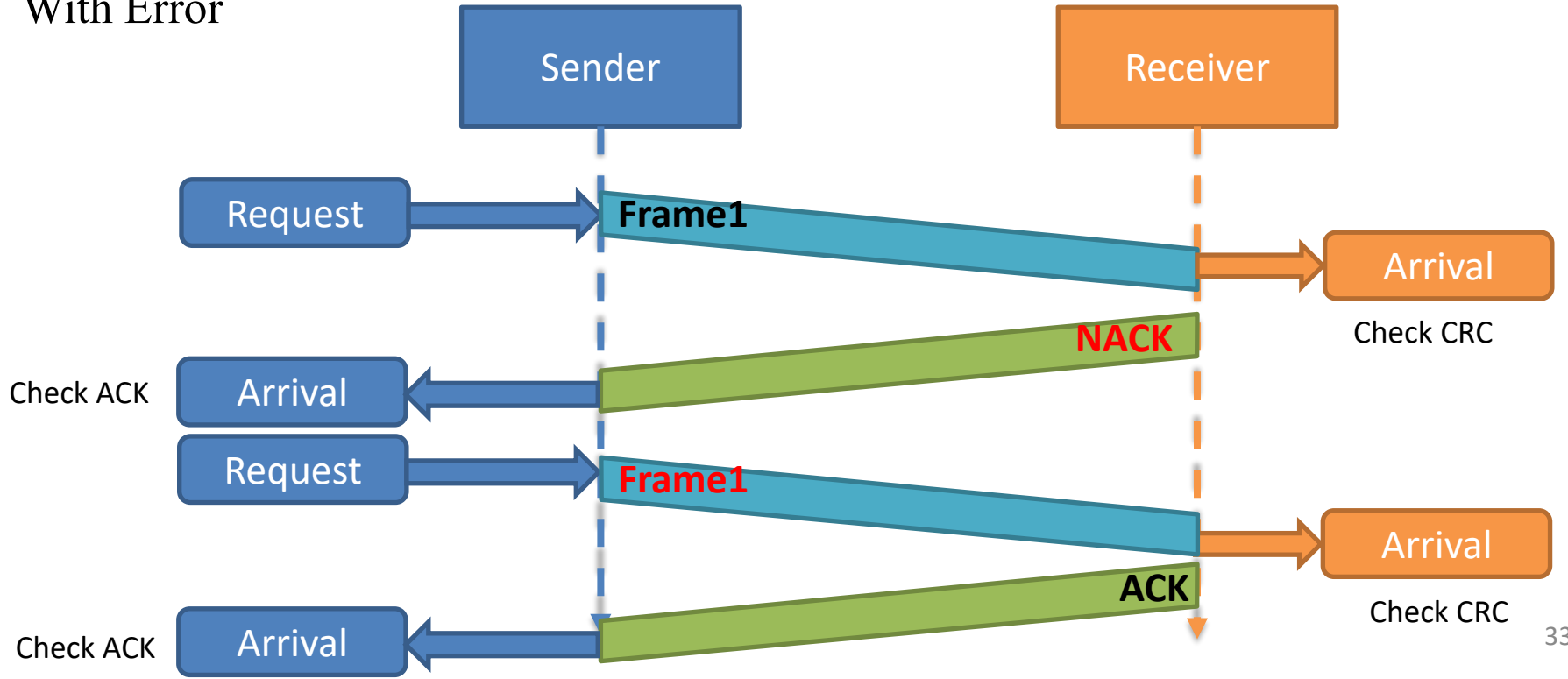
Detecting Frame Corruption

Normal Operation



Detecting Frame Corruption

With Error



Detecting Frame Loss

- ❑ There is still the possibility that **errors** on the channel cause the frames to be **lost entirely**.
- ❑ In particular, the **DLL_ACK** and DLL_NACK frames themselves may be **lost** (or corrupted?) and the sender will be **left waiting forever**.
- ❑ The big question is: *how long should the sender wait for an acknowledgement?*

Detecting Frame Loss

- ❑ To handle these new problems we need to change our *programming paradigm*, from the standard *iterative* one (of C) to an *event-driven* one (as with Java's windowing APIs).
- ❑ Moreover, we'll now need *to handle the concept of time* in our programs and implement protocols which perform nominated actions when *interesting events occur*.

Detecting Frame Loss

❑ In the sender:

```
ESTIMATED_ROUND_TRIP_TIME = 20000 # microseconds

frame = Frame()

def network_layer_ready(): # called if ready
    frame.data = READ_NETWORK_LAYER()
    STOP_NETWORK_LAYER()

    frame.type = FrameType.DLL_DATA
    frame.len = len(frame.data)
    frame.checksum = 0
    frame.checksum = crc16(frame.pack())

    link = 1
    WRITE_PHYSICAL_LAYER(link, frame.pack())
    start_timer(ESTIMATED_ROUND_TRIP_TIME)
```

Detecting Frame Loss

❑ In the sender (cont):

```
def physical_layer_ready(): # frame arrived
    stop_timer()

    ackframe = Frame()
    link, packed = READ_PHYSICAL_LAYER()
    ackframe.unpack(packed)

    if ackframe.type == FrameType.DLL_ACK:
        START_NETWORK_LAYER()
    else:
        WRITE_PHYSICAL_LAYER(link, frame.pack())
        start_timer(ESTIMATED_ROUND_TRIP_TIME)
```

Detecting Frame Loss

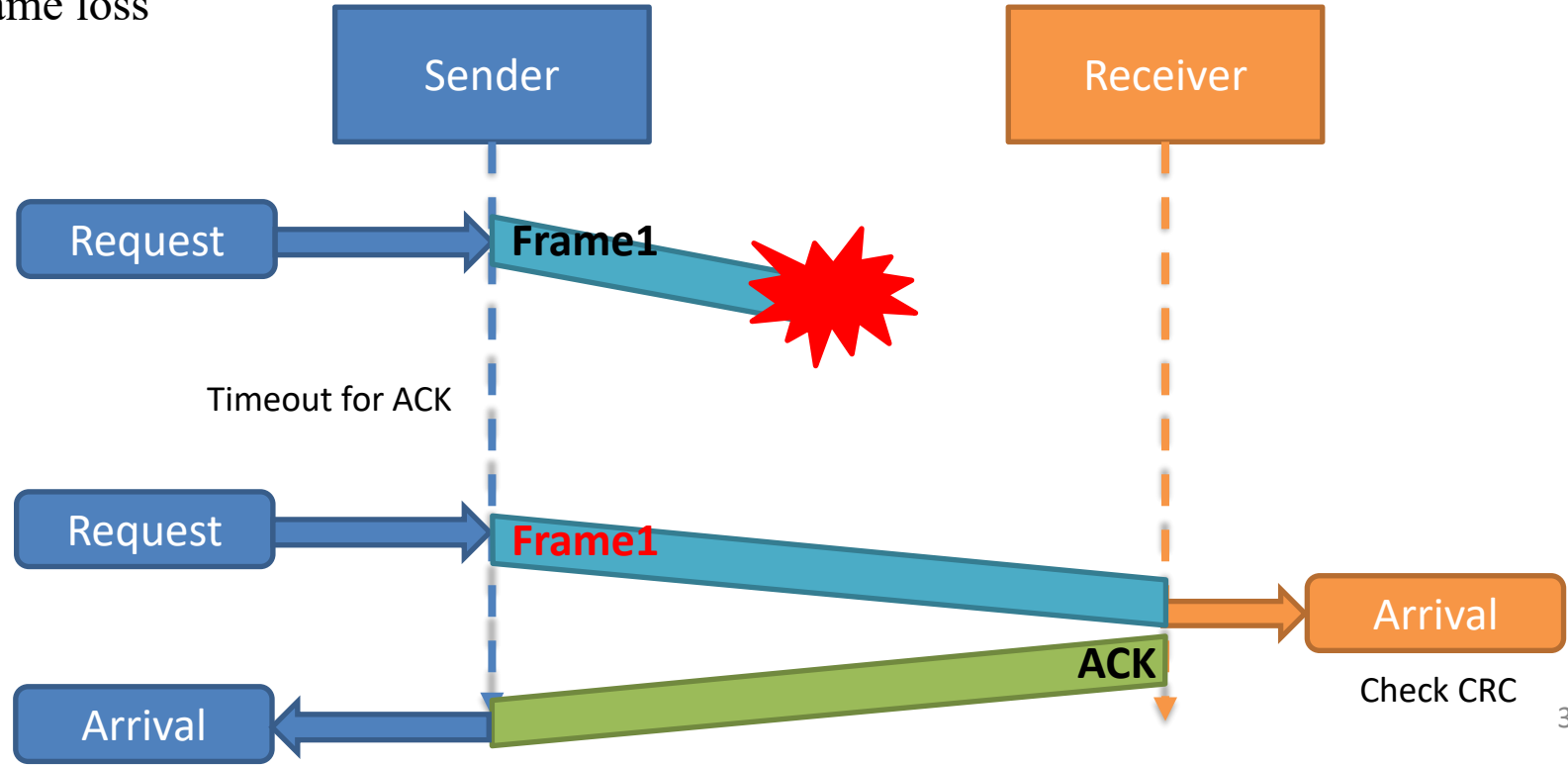
❑ In the sender (cont):

```
def timer_has_expired(): # a timeout
    link = 1
    WRITE_PHYSICAL_LAYER(link, frame.pack())
    start_timer(ESTIMATED_ROUND_TRIP_TIME)
```

And, there is **no need to change** the receiver.

Detecting Frame Loss

With Frame loss



Questions?



Stop-and-Wait Protocol

- ❑ We now present a complete example of the *stop-and-wait* Data-Link layer protocol.
 - This implementation is based on Tanenbaum's (computer networks) 'protocol 4', 5th edition, p230.
 - Some other textbook authors refer to the same protocol as the *ARQ protocol*.
- ❑ This protocol employs **only** data and acknowledgement frames - **piggybacking** and **negative acknowledgements** are not supported.
- ❑ We first define some *global* types, data structures, and variables for this protocol.

Stop-and-Wait Protocol

```
from enum import IntEnum
import struct

from defs import Event
import checksums

class FrameType(IntEnum):
    DLL_DATA = 0
    DLL_ACK = 1
    DLL_NACK = 2

class Frame:
    def __init__(self):
        self.kind = FrameType.DLL_DATA
        self.len = 0      # the length of the msg field only
        self.checksum = 0 # checksum of the whole frame
        self.seq = 0      # only ever 0 or 1
        self.msg = bytes()

    def pack(self):
        return struct.pack('!HHiH{s}'.format(len(self.msg)), self.kind, self.len, self.checksum, self.seq, self.msg)

    def unpack(self, bytes):
        maxbytelen = len(bytes) - struct.calcsize('!HHiH')
        self.kind, self.len, self.checksum, self.seq, self.msg =
            struct.unpack_from('!HHiH{s}'.format(maxbytelen), bytes)
```

Node

```
class Node:
    def __init__(self):
        self.lastmsg = None
        self.lasttimer = None
        self.ackexpected = 0
        self.nextframetosend = 0
        self.frameexpected = 0
```

- ❑ A class named 'Node' must be defined by the **implementation**.
- ❑ One instance of Node is created for **each node** in the network.

Reboot each node

- ❑ We next look at the mandatory `reboot_node()` method.
- ❑ The last 2 lines ensure that data traffic only flows one way, and its acknowledgments only flow the other (which is *much* easier to debug).

Reboot each node

```
def reboot_node(self):
    if (nodeinfo.nodenumbr > 1):
        print('This is not a 2-node network!')
        exit(1)

    set_handler(Event.APPLICATIONREADY, self.application_ready)
    set_handler(Event.PHYSICALREADY, self.physical_ready)
    set_handler(Event.TIMER1, self.timeouts)

    if (nodeinfo.nodenumbr == 0):
        enable_application()
```

Receiving new messages for delivery

- ❑ The first thing of interest that will occur after each node **has rebooted** is that one node's **application layer will generate** and announce a new message for delivery.
- ❑ We handle the Event.APPLICATIONREADY event with our application_ready() method (next slide)

Receiving new messages for delivery

```
def application_ready(self, destination: int, message: bytes):  
    self.lastmsg = message  
    disable_application()  
  
    print('down from application, seq={}'.format(self.nextframetosend))  
  
    self.transmit_frame(self.lastmsg, FrameType.DLL_DATA,  
                        self.nextframetosend)  
  
    self.nextframetosend = 1 - self.nextframetosend
```

Transmit across the physical layer

- ❑ Our `transmit_frame()` method performs the final actions **before** something is transmitted across the Physical layer.
- ❑ Parameters provide the message to be transmitted, an indication as to **whether it is data or an acknowledgment**, and its sequence number as part of the *stop-and-wait* protocol.

Transmit across the physical layer

```
def transmit_frame(self, msg: bytes, kind: FrameType, seqno: int):
    f = Frame()
    f.kind = kind
    f.seq = seqno
    f.checksum = 0
    f.len = len(msg)
    f.msg = msg

    packed = f.pack()
    f.checksum = checksums.checksum_ccitt(packed)
    packed = f.pack()

    link = 1
    write_physical(link, packed)

    if kind == FrameType.DLL_ACK:
        print('{}ACK transmitted, seq={}'.format(self.printspaces, seqno))
    elif kind == FrameType.DLL_DATA:
        print('{}DATA transmitted, seq={}'.format(self.printspaces, seqno))

    timeout = (len(packed) * (8000000 // linkinfo[link].bandwidth)
               + linkinfo[link].propagationdelay)

    self.lasttimer = start_timer(Event.TIMER1, 3 * timeout, None)
```

Handling the arrival of new physical frames

```
def physical_ready(self, linkno: int,
                    framebytes: bytes):

    f = Frame()
    f.unpack(framebytes)

    checksum = f.checksum
    f.checksum = 0
    checksum2 = checksums.checksum_ccitt(f.pack())

    if (checksum != checksum2):
        print('BAD checksum - frame ignored')
        return

    if (f.kind == FrameType.DLL_ACK):
        if (f.seq == self.ackexpected):
            print('ACK received, seq={}'.format(f.seq))
            stop_timer(self.lasttimer)
            self.ackexpected = 1 - self.ackexpected
            enable_application()
```

```
elif (f.kind == FrameType.DLL_DATA):
    if (f.seq == self.frameexpected):
        write_application(f.msg)
        self.frameexpected = 1 -
            self.frameexpected
        result = 'up to application'
    else:
        result = 'ignored'

    print('DATA received, seq={}, {}'.
          .format(f.seq, result))

    self.transmit_frame(bytes(),
                        FrameType.DLL_ACK, f.seq)
```

Improving the stop-and-wait protocol

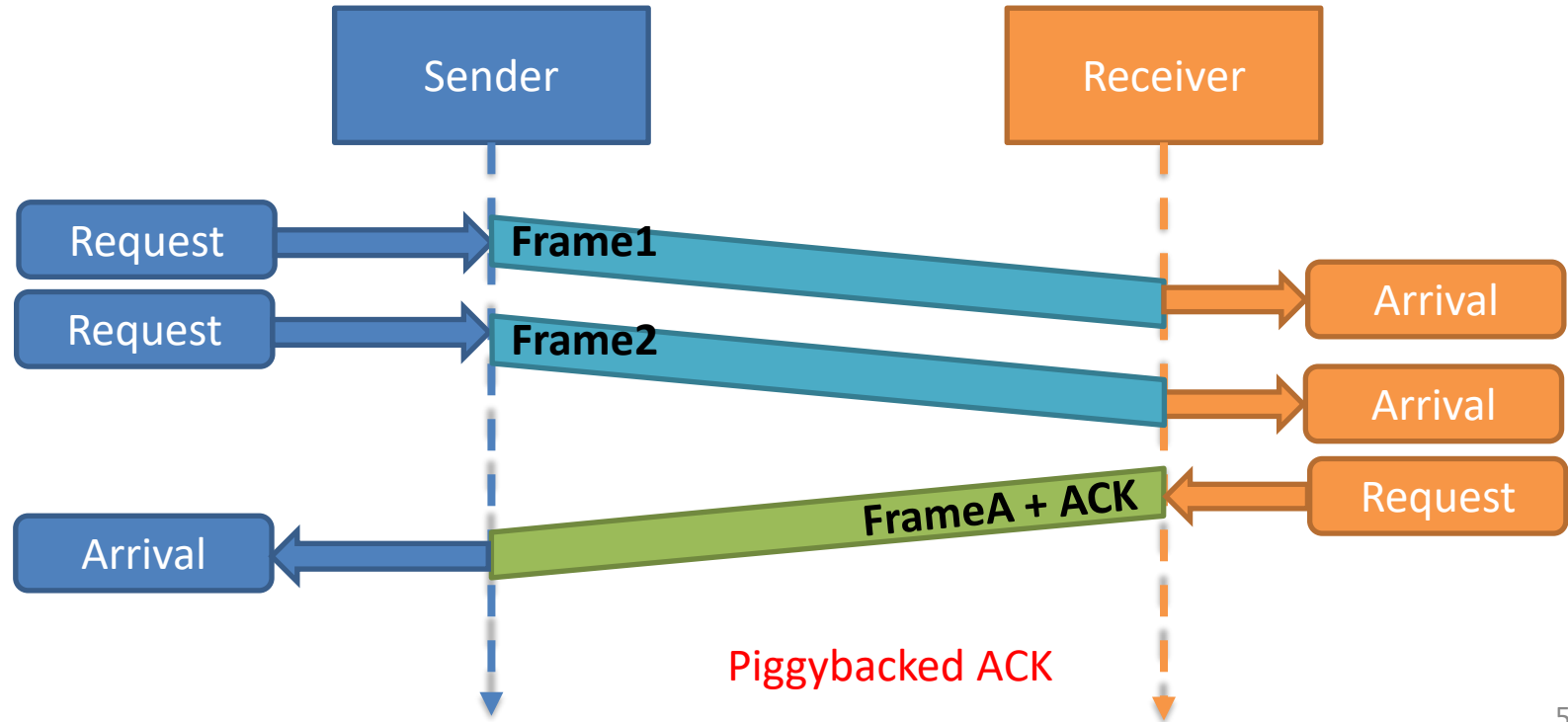
- ❑ We naturally ask 'Can the **stop-and-wait protocol be improved?**'
- ❑ To answer this we first need to **measure** its performance to evaluate potential improvements.
- ❑ Our protocols will always be subject to propagation delays and finite bandwidth - these cannot be overcome. However, our protocols will necessarily:
 - add **extra network traffic** with frame headers,
 - degrade an operating system with **additional interrupts**, and
 - introduce **delays in frame transmission**.
- ❑ Minimizing these overheads will help **improve various efficiencies**.

Improving the stop-and-wait protocol

❑ Reducing the number of data link frames

- At present we have **both** DL_DATA and DL_ACK frames travelling in **each direction**.
- The small DL_ACK frames consume **much bandwidth**, and increase the number of hardware interrupts that must be serviced by the operating system.
- Instead, we use *frame piggybacking*:
 - ❖ When the receiver gets a DL_DATA frame, **it does not immediately send a DL_ACK frame**.
 - ❖ The receiver waits until it has its own outgoing DL_DATA frame, and *piggybacks* the pending DL_ACK in the outgoing header.
 - ❖ If **no DL_DATA** frame becomes available in a short time, the receiver **must send a DL_ACK frame**, by itself.

Improving the stop-and-wait protocol



Sliding Window Protocols

- ❑ The sender **must wait** until an acknowledgment arrives from the receiver.
- ❑ Over links with **long propagation delays** (such as a satellite link with a 540 msec delay) this results in very inefficient use of the available bandwidth.
- ❑ There is thus strong motivation to **keep the sender *and* the medium 'busy'**.
- ❑ We can achieve this by permitting the sender to send **more than a single frame** while **waiting for the first acknowledgment**.

Sliding Window Protocols

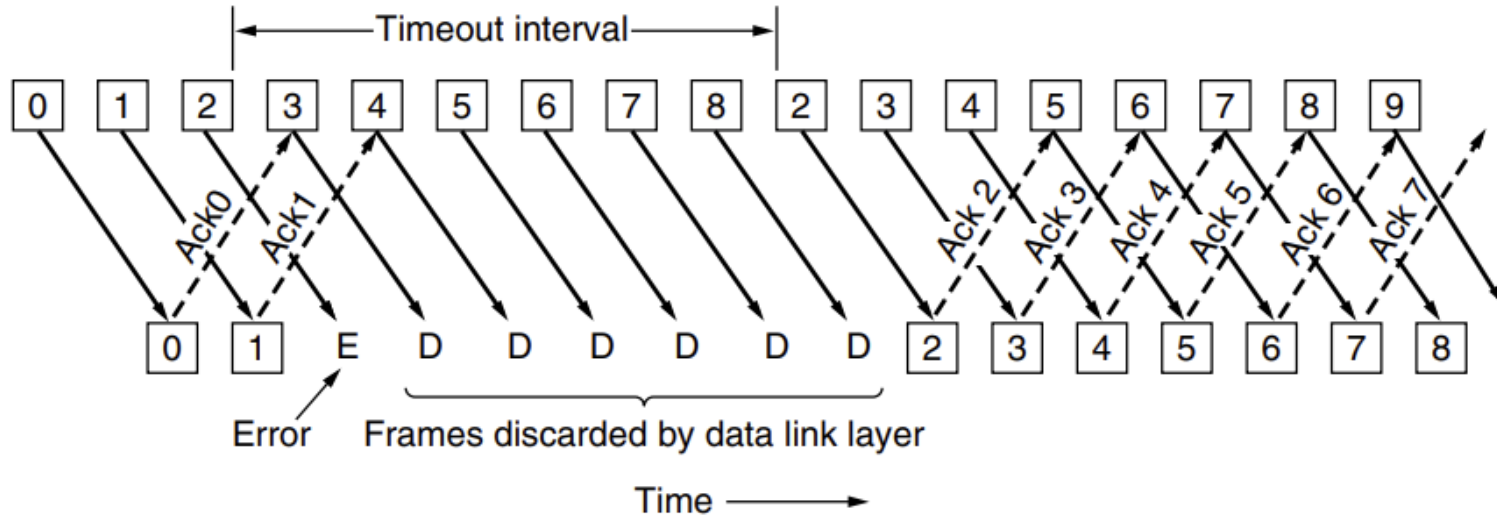
- ❑ In *sliding window* protocols we have these properties:
 - the sender has a **sending window** consisting of a list (array) **of frames** that have been *sent but not acknowledged*.
 - The sender's **window size** grows as more frames are sent but not yet acknowledged.
 - The receiver has **a receiving window** consisting of frames it is *willing to accept*. The receiver's window size remains **constant**.
 - Each frame being sent has **a sequence number** from 0 to $2^n - 1$ (which fits in n bits).
 - Stop-and-wait has $n = 1$.
 - A window size of 1 implies that frames are only accepted in order.

Frame pipelining

- ❑ The obvious question is 'what do we do when *either data frames or acknowledgements are lost?*'
- ❑ We shall look at two solutions:
 - the *go-back-N* protocol
 - the *selective repeat* protocol

Go-Back-N Protocol

- ❑ The first solution, termed *go-back-N*, requires the receiver to simply discard all frames *after* a bad one.
- ❑ This corresponds to a **receiver window size of 1**
- ❑ Sending **no acknowledgements** for the discarded frames



Selective Repeat Protocol

- ❑ The go-back-N protocol **wastes bandwidth** on retransmitted frames if the **error rate is high**.
- ❑ Alternatively, the receiver can **buffer all received frames** (up to some limit) and simply wait for the **bad frame to be retransmitted**.
- ❑ This is termed the **selective repeat protocol** and corresponds to a **receiver window size ≥ 1** .
- ❑ Selective repeat is often combined with having the receiver send a **negative acknowledgement (NAK)** when it detects an error.

Selective Repeat Protocol

